

HW2 Report 110550043

Introduction

這次作業實作了人物骨架的 FK 和 IK。兩個主要的差別在於 FK 是以各關節的角度為輸入，去輸出各關節的位置。IK 則是給定一個目標位置去計算出各關節的角度。

Fundamentals

FK 的基本想法是從局部的資訊出發去推算出整體的位置。意思就是每個 bone 都有他們的 local space，而他們的 world space 則是用他們 parent 的位置 + parent 的朝向 + 自己的 local rotation 去計算出來。簡單來說，是從 root 一節一節往下展開的過程。

IK 的基本想法是從目標資訊出發去反推出需要的局部動作。意思是，當我們知道一個特定的目標位置（例如想要讓手碰到一顆球），我們就必須反向推導每個關節應該要怎麼轉動，才能讓末端（像是手掌）剛好到達這個目標位置。這是一個典型的逆向問題，通常需要用一些數值方法（比如 Jacobian 逆矩陣、CCD、FABRIK 等演算法）來近似求解。簡單來說，就是從末端一路往上調整關節，讓骨架能夠達成目標。

Implementation

FK:

從 root bone 開始，依照 child/sibling 的樹狀結構用 DFS 遞迴處理每個 bone。

對每個 bone，用 parent 的 global rotation、自己的 local base rotation、posture 提供的 local rotation 相乘得到他的 global rotation。

計算每個 bone 的 start position，如果是 root 就用 posture 中的 translation，否則用 parent 的 end position。

計算每個 bone 的 end position，用 $\text{dir} * \text{length} * \text{global rotation}$ 。

IK:

從 end effector 開始，根據目標位置算出 desired vector

反向從 **end effector** 向上傳遞，把誤差對每個 **bone** 的角度調整量計算出來，用 **Jacobian matrix** 來描述對 **end effector** 的影響，再用 **pseudoinverse** 更新計算量。

接著更新每個 **bone** 的 **local rotation**

最後重新用 **FK** 展開一次骨架，讓所有 **bone** 的位置根據新的角度重新計算。

Result and Discussion

● How different step and epsilon affect the result

Step 是迭代時的角度變化量。**step** 太小會導致迭代收斂太慢，太大則會導致無法穩定收斂。

Epsilon 是誤差的範圍。**epsilon** 小結果會更精確，**IK** 會更接近目標，但 **latency** 會較高。**epsilon** 大則反之。

● Touch the target or not

在設定合理的 **step** 和 **epsilon** 後，大部分情況可以讓 **end effector** 接觸到目標。但如果目標超過骨架可以接觸到的範圍或是遇到障礙物時，則只會盡量接近目標沒辦法直接碰到目標。

● Least square solver

在解 **IK** 時用 **pseudoinverse** 求解角度調整量，此方法可以在最小化誤差的同時找到合理的角度變化，讓骨架移動更自然。

Conclusion

這次的作業實作了 **IK** 和 **FK** 的底層程式碼，之前都只有在 **blender** 等工具去拉去拉骨架的 **IK**、**FK**，並沒有真正了解過底層是如何處理這些計算的，透過這次作業學到了不少東西。